

```

        $this->wheels = new
SteelWheels();
    }
    ...
}

```

What's the problem here? Well, the car seems to build its own wheels and engine. While in reality, you should be handing over the engine and the wheels to the car to use. And that way you can give it any engine or wheels you want, without always depending on one concrete implementation like the V8Engine or Steel Wheels. So, your car now asks for the engine and the wheels rather than making them.

```

namespace YourProject\Model;
use Engine;
use Wheels;

public class Car
{
    private $engine;
    private $wheels;
    public function __construct(Engine
$engine, Wheels $wheels)
    {
        $this->engine = $engine;
        $this->wheels = $wheels;
    }
    ...
}

```

To enable this behaviour, you need someone to be able to construct the dependencies and hand it over to the *Car* class. We make use of a

dependency injection container or DIC for this. Some examples are the Auryn and Symfony DIC.

For the purpose of this tutorial, let us use Auryn. You can get it from <https://github.com/rdlowrey/auryn>. It is not absolutely necessary to have a DIC for implementing dependency injection, but at a certain point, it will make your life easier.

To get started, simply ask it to give you an instance of *Car* and it will recursively use reflection to build the dependencies. In case your dependencies have a complex construction mechanism, you can delegate the constructor to a factory method.

```

$injector = new Auryn\Injector;
$injector->alias('Engine', 'V8Engine');
$injector->alias('Wheels',
'SteelWheels');

$car = $injector->make('Car');

```

Here, *Engine* and *Wheels* are interfaces and not classes. In the above snippet, I am asking Auryn to build the concrete implementations V8Engine and SteelWheels, where I have specified *Engine* and *Wheels* as dependencies. We bind the interfaces to their concrete implementations using the alias method. This way, we

can decouple classes from concrete implementations.

A DIC can also enable you to have a single instance of a class to be shared across objects, with ease; for example, a database connection with costly construction.

```
$injector = new Auryn\Injector;
```

```
$injector->share('Engine');
```

Now every time you require *Engine* as a dependency, you will get the same instance. This is useful only in certain cases; so make sure that you really need one single instance of a class. In most cases, you don't.

Your code is already scalable! You have decoupled things to a certain extent, and separated the concerns, so you can easily add and delete modules as you need.

In this tutorial, we have laid the foundation for building clean PHP applications and, as a matter of fact, we can apply the same principles to any other language.

In the next article in this series, we will look at how we can break up our project further into independent modules, enable templating and routing, as well as make full use of the powerful language that PHP is. **END** 

 By: Dedipyaman Das

The author is a software developer and FOSS contributor.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU